

LEGAL EASE
AI-POWERED LEGAL DOCUMENT
GENERATOR
PROJECT DOCUMENTATION

Pre-development / Planned Implementation

Definition

LegalEase is an AI-powered legal document generation platform designed to simplify the creation of commonly required legal documents. The system collects relevant information from users, processes the input using generative artificial intelligence, and produces a structured legal-document draft.

The proposed platform supports document types such as employment contracts, Non-Disclosure Agreements (NDAs), lease agreements, and other agreements. It combines a user-friendly interface, AI-based generation, document preview and editing, professional formatting, and export to TXT, DOCX, and PDF formats.

This document describes the proposed system and development plan. It does not claim that the complete application has already been implemented, deployed, or hosted.

Team Members Name & Roles:

- 1. Name: Shaarika M (Team Leader) Role: Coding, Voice over
- 2. Name: Shiny Alice S (Team Member) Role: Documentation
- 3. Name: Saranya V (Team Member) Role: APT Video
- 4. Name: P. Pandimeena (Team Member) Role: Demo Video

1.0 Executive Summary & Project Overview

LegalEase is proposed as an AI-powered platform that reduces the manual effort involved in preparing structured legal documents. Users would provide essential information such as document type, parties, terms and effective dates, and the system would use that information to prepare an organized legal-document draft.

The planned architecture combines Streamlit for the user interface, FastAPI for backend request handling, Gemini for AI-based generation, and document-processing components for TXT, DOCX, and PDF outputs.

1.1 Core Objectives

- Generate structured legal-document drafts from user-provided information.
- Allow users to specify document type, parties, terms, and effective dates.
- Use Gemini-based generative AI to organize supplied information into a document structure.
- Provide preview and editing before export.
- Support TXT, DOCX, and PDF output.
- Provide structured formatting such as headings, tables, logos, and footers where applicable.

1.2 Key Platform Capabilities

Capability	Purpose
Document Type Selection	Identify the required legal-document category.
Structured Input	Collect parties, terms, dates, and document-specific information.
AI Generation	Generate an organized legal-document draft.
Preview & Editing	Review and modify generated content.
Document Formatting	Prepare structured output.
Multi-Format Export	Provide TXT, DOCX, or PDF.

2.0 Scenario-Based Case Study & Requirements

2.1 Background & Motivation

Manual legal-document preparation can require users to understand document structures, organize terms, and maintain consistent formatting. LegalEase is proposed to simplify these activities through structured input, AI-assisted generation, and a review-and-export workflow.

2.2 Problem Statement

Problem Area	LegalEase Approach	Expected Benefit
Manual drafting	AI-assisted structured generation	Reduces repetitive drafting effort.
Complex document structure	Guided input fields	Helps users provide relevant information.
Different document types	Document-type selection	Supports multiple categories.
Formatting effort	Automated formatting	Improves consistency of outputs.
Need for review	Preview and editing	Keeps the user involved before export.

2.3 Use Case Scenarios

1. Employment Contract — user supplies employment-related details and receives a structured draft for review.
2. Non-Disclosure Agreement — user supplies parties, confidentiality terms, and effective date and receives a structured NDA draft.
3. Lease Agreement — user supplies landlord/tenant and lease-related details and receives a structured lease draft.
4. Other Agreement — the workflow can be extended to additional supported agreement types.

2.4 Software Requirements

Component	Planned Technology
Language	Python 3.10+
Frontend	Streamlit
Backend	FastAPI
AI	Google Generative AI / Gemini
DOCX	python-docx
PDF	FPDF
Supporting	Pillow, requests, python-dotenv

2.5 Hardware Requirements

- Standard laptop or desktop capable of running Python and a local web application.
- Stable internet connection for the external generative-AI service.
- Sufficient storage for dependencies and generated files.

3.0 System Architecture & Design Patterns

The proposed LegalEase architecture separates user interaction, backend request handling, AI generation, document processing, and output generation into distinct logical components.

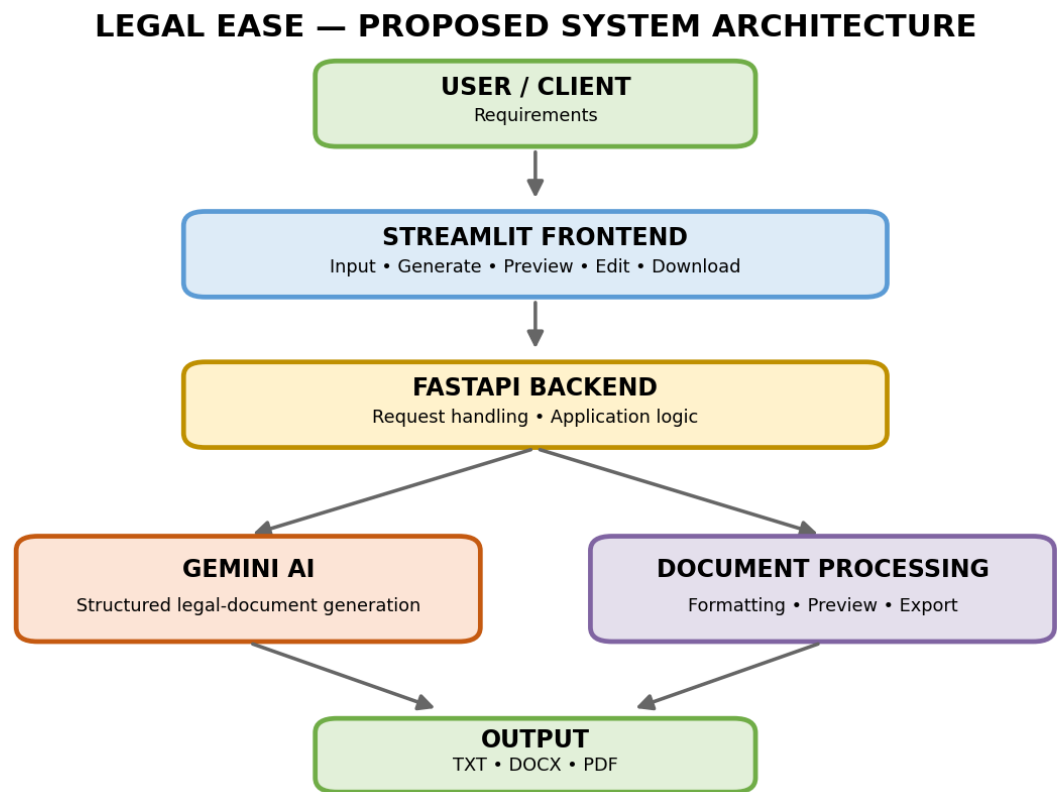


Figure 3.1: Proposed LegalEase system architecture

3.1 Architecture Components

Component	Responsibility
User / Client	Provides document requirements and reviews the draft.
Streamlit Frontend	Input form, generation controls, preview, editing, and downloads.
FastAPI Backend	Receives requests and coordinates processing.
Gemini AI Layer	Generates structured legal-document content.
Document Processing	Formats and prepares TXT, DOCX, and PDF outputs.
Preview / Editing	Supports review and user modification.
Output Layer	Provides the selected final format.

3.2 Architectural Flow

5. User enters document requirements.
6. Streamlit collects document type, parties, terms, and effective date.
7. FastAPI receives the request.
8. The generation workflow prepares structured input for Gemini.
9. Gemini generates the legal-document content.
10. Document processing prepares the content for preview and export.
11. The user reviews or edits the draft and selects TXT, DOCX, or PDF output.

4.0 Data Flow & Functional Architecture

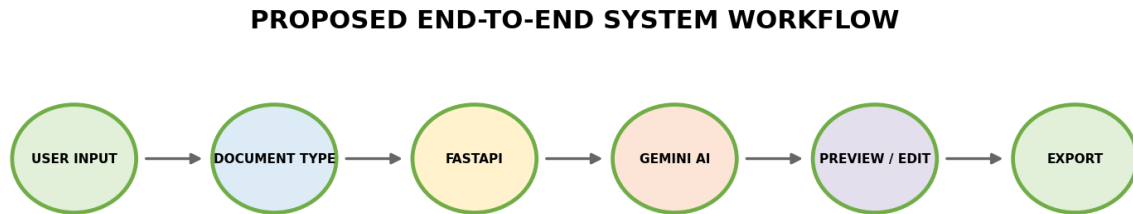


Figure 4.1: Proposed LegalEase data flow

4.1 Input Data

- Document type
- Parties involved
- Terms and conditions
- Effective date
- Other document-specific information

4.2 AI Processing

The collected information is intended to be organized into a structured generation request. The Gemini component then produces legal-document content based on the supplied requirements.

4.3 Output Processing

Generated content is intended to pass through preview and editing before the user selects the required output format. The project source identifies TXT, DOCX, and PDF as the planned output formats.

5.0 Core Functional Modules

CORE COMPONENT MAP

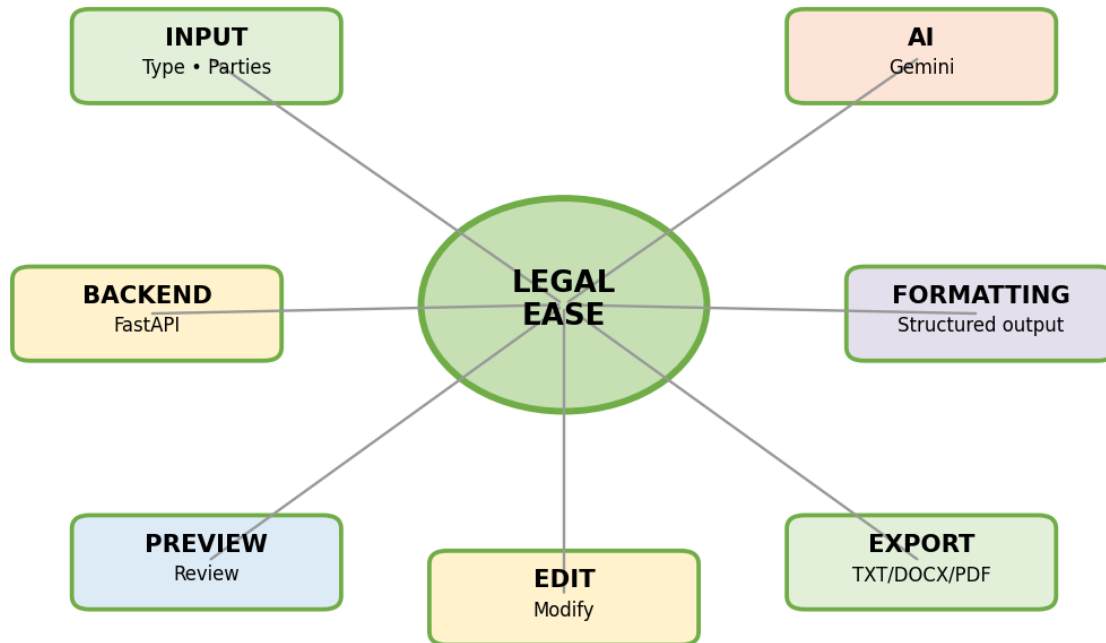


Figure 5.1: Proposed LegalEase core component map

5.1 User Input Module

Collects document type, parties, terms, effective date, and other relevant information.

5.2 AI Document Generation Module

Uses Gemini-based generation to transform structured input into legal-document content.

5.3 Document Formatting Module

Prepares structured outputs and supports formatting elements such as headings, tables, logos, and footers.

5.4 Preview & Editing Module

Provides a review stage before the final document is exported.

5.5 Export Module

Prepares TXT, DOCX, and PDF versions of the generated document.

6.0 End-to-End System Workflow

PROPOSED END-TO-END SYSTEM WORKFLOW

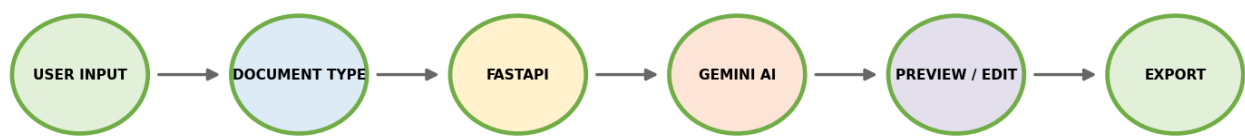


Figure 6.1: Proposed LegalEase workflow

Stage	Process	Expected Result
1	Enter requirements	Structured input
2	Select document type	Generation context
3	Submit request	Backend request
4	AI generation	Legal-document draft
5	Preview	Reviewable content
6	Edit if required	User-adjusted draft
7	Export	TXT / DOCX / PDF

7.0 Project Epics & Development Phases

PROPOSED PROJECT DEVELOPMENT ROADMAP

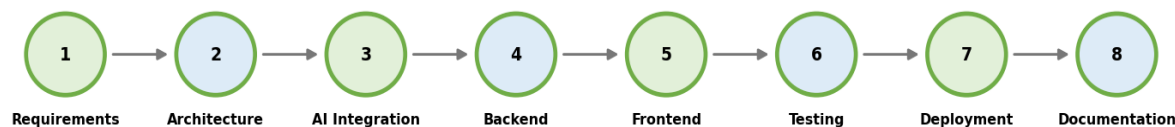


Figure 7.1: Proposed development roadmap

Epic 1: Requirements & Scope

Define supported documents, inputs, outputs, objectives, and workflow.

Epic 2: Architecture Design

Define Streamlit, FastAPI, Gemini, and document-processing interactions.

Epic 3: AI Integration

Prepare the generation workflow and Gemini connection.

Epic 4: Backend Development

Develop FastAPI request handling and generation services.

Epic 5: Frontend Development

Create input, generation, preview, editing, and download interfaces.

Epic 6: Document Processing

Implement TXT, DOCX, PDF generation and formatting.

Epic 7: Testing & Deployment

Validate the workflow and prepare hosting/deployment.

Epic 8: Documentation

Maintain README, diagrams, setup instructions, and final documentation.

8.0 Security & Responsible AI Considerations

- Store API keys through environment variables rather than source code.
- Validate user inputs before processing.
- Handle AI-service failures and invalid inputs safely.
- Allow users to review and edit generated content before export.
- Avoid exposing sensitive information during development and testing.
- Clearly communicate that generated documents may require professional legal review.

8.1 Error Handling

- Missing or incomplete fields
- AI service failure
- Document formatting failure
- Unexpected or invalid input
- User-readable error messages

9.0 Testing Strategy

Testing Area	Planned Check
Input Testing	Required and invalid fields.
AI Generation Testing	Supported document types and varied inputs.
API Testing	Frontend ↔ FastAPI communication.
Output Testing	TXT, DOCX, and PDF generation.
Formatting Testing	Headings, tables, logos, and footers where applicable.
Integration Testing	Complete input → AI → output workflow.
Error Testing	Service and validation failures.
User Acceptance Testing	Whether users can complete the intended workflow.

These are planned validation activities. Actual test results and deployment evidence should be added after implementation.

10.0 Expected Output

The proposed system is expected to generate a structured legal-document draft based on the selected document type and user-provided information.

Output	Purpose
TXT	Plain-text document content.
DOCX	Editable Word document with structured formatting.
PDF	Formatted document for sharing or printing.

10.1 Expected Document Types

- Employment Contract
- Non-Disclosure Agreement (NDA)
- Lease Agreement
- Other supported agreements

11.0 Advantages

- Reduces repetitive effort in preparing structured document drafts.
- Provides a guided workflow for users unfamiliar with legal-document structures.
- Uses AI to organize supplied information.
- Provides preview and editing before export.
- Supports multiple output formats.
- Provides a modular foundation for future document types.

11.1 Limitations

- AI-generated content may require human review.
- Legal requirements can vary by jurisdiction and document type.
- The system should not replace professional legal advice.
- AI-service availability and response quality can affect generation.
- Output quality depends partly on the information supplied by the user.

12.0 Future Enhancements

- Additional legal-document types.
- Deeper contract analysis.
- Integration with legal databases.
- Personalized recommendations.
- Expanded document-processing capabilities.
- Additional formatting and customization options.

13.0 GitHub & Development Plan

After the documentation and architecture are finalized, the project can be organized as a GitHub repository containing the application source code, dependency configuration, documentation, environment-variable instructions, and project assets.

Repository Area	Purpose
Frontend	Streamlit interface.
Backend	FastAPI services and application logic.
AI Integration	Gemini generation workflow.
Document Processing	TXT, DOCX, PDF generation.
Documentation	README, diagrams, setup and project documentation.
Configuration	Example environment configuration without exposing secrets.

The documentation should be updated as implementation progresses so that the written design, GitHub repository, and final application remain consistent.

14.0 Conclusion

LegalEase is proposed as an AI-powered legal document generation platform that combines structured user input, generative AI, document processing, preview and editing, and multi-format export into one workflow.

The planned architecture separates the Streamlit interface, FastAPI backend, Gemini AI generation, and document-processing components so that each area can be developed and tested independently.

The project documentation provides the foundation for development from requirements and architecture through implementation, testing, GitHub organization, deployment, and final documentation. Human review remains important because AI-generated legal content can require correction or professional validation.